

tmech: A C++ header-only library for compile-time tensor algebra with symbolic differentiation

Peter Lenz ¹

¹ Continental Reifen Deutschland GmbH

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 15 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

tmech is a C++17 header-only library for higher-order tensor algebra aimed at computational continuum mechanics. It provides a high-level, coordinate-free syntax for tensors of arbitrary rank and an expression-template engine that composes operations without intermediate temporaries. Beyond standard tensor algebra, it offers spectral-based isotropic tensor functions (exp, log, sqrt, pow) and tensor decompositions (polar, eigenvalue) with closed-form derivatives, and an in-development compile-time *symbolic differentiation* module that produces analytic tangent operators for tensor-valued expressions. Tensors up to rank eight (in three dimensions) use stack-allocated storage; larger ranks fall back to dynamic allocation, and optional SIMD acceleration is available through xsimd ([QuantStack, 2023](#)). The library targets nonlinear solid and fluid mechanics, where consistent linearization of constitutive models is central to Newton-type finite-element solvers.

Statement of need

Computational continuum mechanics places two demands on a numerical library. First, constitutive models are expressed in higher-order tensor algebra — symmetry-aware products, contractions, isotropic tensor functions, and spectral or polar decompositions of second- and fourth-order tensors. Second, Newton-type solvers for the resulting nonlinear boundary-value problems require *consistent tangent operators* — fourth-order tensors obtained by differentiating the stress response with respect to a deformation measure ([Bonet & Wood, 2008](#); [Holzapfel, 2000](#)) — whose derivation by hand is tedious, error-prone, and a recurring bottleneck when developing new material models ([Simo, 1992](#)).

Existing tools address at most one of these. General C++ tensor libraries provide efficient evaluation but not both higher-order continuum-mechanics operations and differentiation: Eigen ([Guennebaud et al., 2010](#)) targets matrices and vectors rather than higher-order tensors, while FTensor ([Landry, 2003](#)) and Fastor ([Poya et al., 2017](#)) offer higher-order tensor contractions through expression templates but no differentiation. Conversely, general-purpose automatic differentiation tools — Sacado ([Heroux & others, 2005](#)), CoDiPack ([Sagebaum et al., 2019](#)), autodiff ([Leal, 2018](#)), and Enzyme ([Moses & Churavy, 2020](#)) — differentiate arbitrary C++ code but produce derivative *values* at run time and are not specialized to tensor algebra, and symbolic code generators such as AceGen ([Korelc, 2002](#)) emit closed-form code but depend on an external computer-algebra environment.

tmech combines both in a single header-only library: a tensor-algebra engine for arbitrary-rank tensors with the operations continuum mechanics needs, together with an in-development compile-time symdiff module that differentiates tensor expressions symbolically — entirely within C++, yielding closed-form derivative tensors in the library's own types without a run-time tape or external toolchain. The tensor-algebra engine is production-tested: it underlies the author's multiscale composite simulations ([Lenz & Mahnken, 2023, 2024](#)) and is a dependency

of the numsim-cas (NumSim-Stack, 2026a) and numsim-codegen (NumSim-Stack, 2026b) tools.

Functionality

tmech uses expression templates with the curiously recurring template pattern (Coplien, 1995; Veldhuizen, 1995): tensor operations — contractions, products, inverse, transpose, deviatoric/volumetric projections, and decompositions — assemble a compile-time expression tree that is evaluated only on assignment, avoiding temporaries. The symdiff module extends this to symbolic algebra, so that a consistent tangent operator can be obtained directly from a stress definition. For a linear-elastic model:

```
using T2 = tmech::tensor<double,3,2>;
symdiff::variable<T2, 0> eps;
symdiff::constant<double, 0> lambda(150, "lambda");
symdiff::constant<double, 1> mu(200, "mu");
symdiff::real<double, 2, 0, 1> two;
symdiff::constant<T2, 2> I("I"); I = tmech::eye<double,3,2>();

auto sig = lambda * tmech::trace(tmech::as_sym(eps)) * I
          + two * mu * tmech::as_sym(eps); // stress
auto C    = symdiff::derivative(sig, eps); // tangent d(sigma)/d(eps)
```

The same mechanism yields the stress of a hyperelastic model by differentiating its strain-energy function. Chaining both steps into a single energy-to-tangent differentiation, and support for models that require implicit (iterative) integration such as inelastic materials, are in development.

Isotropic tensor functions and polar/eigenvalue decompositions additionally expose analytic derivative tensors (Miehe, 1993; Miehe & Lambrecht, 2001), required when constitutive models are formulated in terms of stretch or rotation tensors. The library is header-only, requires only a C++17 compiler (GCC 7+, Clang, MSVC) with no mandatory dependencies, and is tested with Google Test across GCC, Clang, and MSVC on Linux, macOS, and Windows. Further examples and API documentation are provided in the repository.

Performance

tmech is benchmarked against Eigen (Guennebaud et al., 2010) and Fastor (Poya et al., 2017) using Google Benchmark (Google, 2023). Figure 1 shows representative operations: tmech is consistently faster than Eigen (up to about a factor of three) and competitive with Fastor — faster on most operations and within a small factor on the rest. Benchmark sources and full reports are provided in the repository.

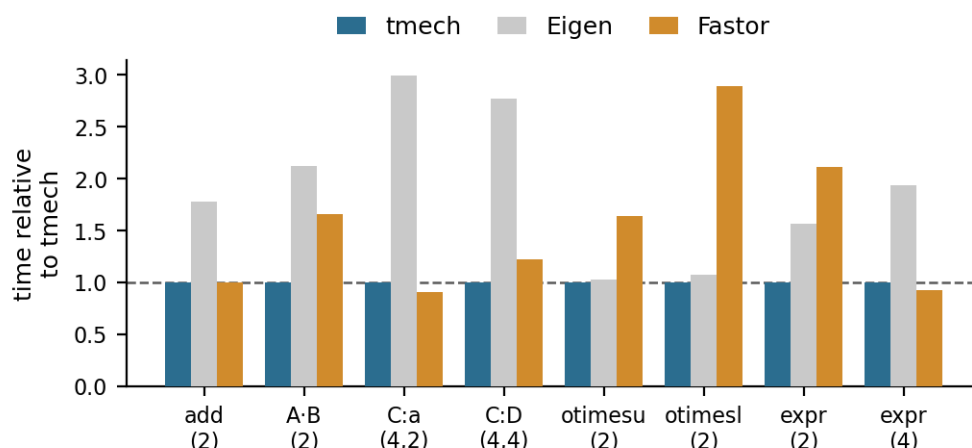


Figure 1: Execution time on representative tensor operations, relative to tmech (lower is faster; SIMD build via xsimd 13.2.0, GCC 13.3 with `-O3 -march=native -funroll-loops`, on an AMD Ryzen 7 PRO 6850U, Ubuntu 24.04 / Linux 6.17, single machine; identical flags for tmech, Eigen 3.4.0 and Fastor (commit a300234); timings via Google Benchmark 1.9.1). $A \cdot B$ is a single contraction; $C:a$ and $C:D$ are fourth-order double contractions with a second- and fourth-order tensor, respectively; `expr` is a compound expression.

AI usage disclosure

Generative AI (Claude Opus 4.8, Anthropic, via Claude Code) assisted with this submission: reviewing and fixing library code, extending the test suite and CI, drafting documentation, and editing this manuscript. The core design and algorithms are the author's own. All AI-assisted content was reviewed and validated by the author, who made all design decisions and takes full responsibility for the submission.

Acknowledgements

The author thanks the open-source communities behind Eigen, xsimd, and Google Benchmark.

References

- Bonet, J., & Wood, R. D. (2008). *Nonlinear Continuum Mechanics for Finite Element Analysis* (2nd ed.). Cambridge University Press. <https://doi.org/10.1017/CBO9780511755446>
- Coplien, J. O. (1995). Curiously recurring template patterns. *C++ Report*, 7(2), 24–27.
- Google. (2023). *Benchmark: A microbenchmark support library*. <https://github.com/google/benchmark>
- Guennebaud, G., Jacob, B., & others. (2010). *Eigen v3*. <https://eigen.tuxfamily.org>
- Heroux, M. A., & others. (2005). An overview of the Trilinos project. *ACM Transactions on Mathematical Software*, 31(3), 397–423. <https://doi.org/10.1145/1089014.1089021>
- Holzappel, G. A. (2000). *Nonlinear Solid Mechanics: A Continuum Approach for Engineering*. John Wiley & Sons. ISBN: 978-0471823193
- Korelc, J. (2002). Multi-language and multi-environment generation of nonlinear finite element codes. *Engineering with Computers*, 18(4), 312–327. <https://doi.org/10.1007/s003660200028>

- 89 Landry, W. (2003). Implementing a high performance tensor library. *Scientific Programming*,
90 11(4), 273–290. <https://doi.org/10.1155/2003/693153>
- 91 Leal, A. M. M. (2018). *Autodiff: Automatic differentiation made easier for C++*. <https://github.com/autodiff/autodiff>
- 92
- 93 Lenz, P., & Mahnken, R. (2023). Non-local integral-type damage combined to mean-field
94 homogenization methods for composites and its parallel implementation. *Composite*
95 *Structures*, 314, 116911. <https://doi.org/10.1016/j.compstruct.2023.116911>
- 96 Lenz, P., & Mahnken, R. (2024). Multiscale simulation of polymer curing of composites
97 combined mean-field homogenisation methods at large strains. *International Journal of*
98 *Solids and Structures*, 290, 112642. <https://doi.org/10.1016/j.ijsolstr.2023.112642>
- 99 Miehe, C. (1993). Computation of isotropic tensor functions. *Communications in Numerical*
100 *Methods in Engineering*, 9(11), 889–896. <https://doi.org/10.1002/cnm.1640091105>
- 101 Miehe, C., & Lambrecht, M. (2001). Algorithms for computation of stresses and elasticity
102 moduli in terms of Seth–Hill's family of generalized strain tensors. *Communications in*
103 *Numerical Methods in Engineering*, 17(5), 337–353. <https://doi.org/10.1002/cnm.404>
- 104 Moses, W. S., & Churavy, V. (2020). Instead of rewriting foreign code for machine learning,
105 automatically synthesize fast gradients. *Advances in Neural Information Processing Systems*
106 (*NeurIPS*), 33, 12472–12485.
- 107 NumSim-Stack. (2026a). *Numsim-cas*. <https://github.com/NumSim-Stack/numsim-cas>
- 108 NumSim-Stack. (2026b). *Numsim-codegen*. <https://github.com/NumSim-Stack/numsim-codegen>
- 109
- 110 Poya, R., Gil, A. J., & Ortigosa, R. (2017). A high performance data parallel tensor contraction
111 framework: Application to coupled electro-mechanics. *Computer Physics Communications*,
112 216, 35–52. <https://doi.org/10.1016/j.cpc.2017.02.016>
- 113 QuantStack. (2023). *Xsimd: C++ wrappers for SIMD intrinsics*. <https://github.com/xtensor-stack/xsimd>
- 114
- 115 Sagebaum, M., Albring, T., & Gauger, N. R. (2019). High-Performance Derivative
116 Computations using CoDiPack. *ACM Transactions on Mathematical Software*, 45(4).
117 <https://doi.org/10.1145/3356900>
- 118 Simo, J. C. (1992). Algorithms for static and dynamic multiplicative plasticity that preserve the
119 classical return mapping schemes of the infinitesimal theory. *Computer Methods in Applied*
120 *Mechanics and Engineering*, 99(1), 61–112. [https://doi.org/10.1016/0045-7825\(92\)90123-2](https://doi.org/10.1016/0045-7825(92)90123-2)
- 121
- 122 Veldhuizen, T. L. (1995). Expression templates. *C++ Report*, 7(5), 26–31.